SigLIP-SO400M — Complete Architecture, Rationale and Defense Q&A

FoodGenome AI — Supplementary Technical Note

1 What SigLIP Actually Is

SigLIP = Sigmoid Loss for Language-Image Pre-training. It is Google's successor to CLIP. Both are *vision–language* models: they learn by matching images with their text captions rather than by learning to predict a fixed set of class labels.

The key innovation — sigmoid loss instead of softmax loss

- **CLIP:** for each image in a batch, a softmax is computed over *all* captions in that batch to decide which one matches. The loss for a single image therefore depends on every other image in the batch, which forces very large batch sizes (often 32k+) and expensive batch-wide normalization.
- **SigLIP:** treats every (image, text) pair independently as a binary "match / no match" decision via a sigmoid function. No batch-wide normalization is required, which lets it scale to large batches cheaply and train efficiently even with modest batch sizes.

2 Full Architecture (as used in this project)

The backbone used is `vit_so400m_patch14_siglip_384.webli` — a plain Vision Transformer image encoder. Only the *image tower* was used (not the text tower), since classification is performed by a probe head trained on top of frozen embeddings.

Pipeline for one image

```
Input image (384x384x3)
     |
     v  Patchify: cut into 14x14 pixel patches
27 x 27 = 729 patches
     |
     v  Linear projection: each patch (14x14x3=588 numbers) -> 1152-dim vector
     |
     v  Add positional embeddings (encodes patch location)
     |
     v  27 x Transformer blocks, each:
        +----------------------------------+
        | LayerNorm                        |
        | Multi-Head Self-Attention (16h)  |--+
        |        + residual  <-------------+--+
        | LayerNorm                        |
        | MLP (1152 -> 4608 -> 1152)       |--+
        |        + residual  <-------------+--+
        +----------------------------------+
     |
```

```
     v  Pool 729 token vectors into a single 1152-dim embedding
Output: 1152-d feature vector
```

Self-attention (core computation of every block)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

where Q, K, V are linear projections of the token embeddings. This lets every patch "look at" every other patch and decide how much information to borrow from each.

Specifications

Property	Value
Parameters	428.2M (report figure; "400M" in the name is the nominal design target)
Transformer blocks	27
Embedding dimension	1152
Attention heads	16
Patch size	14×14
Input resolution	384 px
Tokens per image	729
Pretraining data	WebLI (web-scale image–text pairs)

"SO" = Shape-Optimized

Ordinary ViTs scale width, depth, and MLP ratio by fixed conventions (e.g. standard ViT-L, ViT-H recipes). SigLIP's authors instead ran a small architecture search over these three dimensions to find the most *compute-efficient* shape — which is why SO400M has an unusual depth (27, not 24) and width (1152, not 1024). It is not an off-the-shelf ViT ratio; it has been tuned for performance per FLOP.

3 Why This Model Was Chosen

Two independent conditions both had to hold.

Condition A — it had to run on the available hardware

Apple M3 Pro, 18 GB unified memory, no CUDA. A ViT-g or InternVL-scale model (2B+ parameters) does not fit in memory even for inference-only. SigLIP at 428M parameters in float16 is ≈0.9 GB — comfortably runs.

Condition B — it had to be the strongest feature extractor at that size

SigLIP alone, with just a 3.7-minute linear probe, beat a multi-hour full fine-tune of EVA-02-L (95.90%). This is strong evidence that SigLIP's contrastive image–text pretraining already encodes "what food looks like" well enough that a linear read-out on frozen features is sufficient — no weight updates needed.

Model	Test top-1 (probe)
DINOv2-L	94.87%
EVA-02-L	95.53%
SigLIP-SO400M	**96.83%**

4 Why Not the Alternatives

Candidate	Why rejected
ResNet-50 / EfficientNet	CNNs plateau around 85–90% on Food-101; would make the reliability/calibration work the only novel contribution.
ViT-B/16 (86M)	Runs easily, but measurably weaker on fine-grained food categories.
ViT-g / InternVL (2B+)	Does not fit in 18 GB unified memory even for inference.
CLIP (instead of SigLIP)	Needs large-batch softmax training with global normalization — more expensive and less favorable at this scale; SigLIP's sigmoid loss gets better results per parameter.
DINOv2-L alone	Weaker (94.87%) — self-supervised, no language signal; good at texture/parts but not semantic category discrimination.
EVA-02-L alone	Strong (95.53–95.90%) but still behind SigLIP; kept as an ensemble partner rather than a replacement.

5 Why Three Backbones — and Why SigLIP Is the Anchor

Three *methodologically different* pretraining objectives were chosen deliberately, since ensembling only helps when models fail on *different* images.

Model	Pretraining signal	What it is best at
SigLIP	Image ↔ text (contrastive)	"What is this thing" — semantic identity
DINOv2	No labels at all (self-distillation)	Texture, parts, structure
EVA-02	Masked-image-modelling + supervised	Fine-grained discrimination

Empirical proof the diversity mattered

SigLIP and EVA-02 agree on 94.42% of test images; SigLIP alone gets an extra 2.40% right, EVA-02 alone gets an extra 1.10% right. The *oracle* accuracy (either model correct) reaches 98.30% — the 1.14-point gap above the actual 97.16% ensemble result is the headroom ensembling is chasing.

DINOv2 was tested as a third ensemble member but *excluded* from the final model: at 94.87% it is the weakest member, and in a uniform average a weaker member drags the mean down more than its decorrelated errors recover.

6 Transformer Basics — Self-Attention, Encoder, Decoder

This section explains the general Transformer building blocks that SigLIP (and every other backbone in this project) is built from. Answers are given in plain question/answer form.

Q: What is a Transformer, in one sentence?

A neural network architecture that processes a sequence of tokens (words, or image patches) by letting every token exchange information with every other token through *self-attention*, instead of processing them one at a time (like RNNs) or only looking at nearby neighbours (like CNNs).

Q: What is self-attention and how does it work step by step?

Self-attention lets each token decide, for itself, how much to "listen to" every other token in the sequence.

1. Every token's embedding vector x_i is linearly projected into three separate vectors:

$$Q = XW_q \quad \text{(Query — "what am I looking for?")}$$

$$K = XW_k \quad \text{(Key — "what do I contain?")}$$

$$V = XW_v \quad \text{(Value — "what information do I pass on?")}$$

2. Every query is compared against every key with a dot product: $QK^\top$. This produces a similarity score between every pair of tokens.

3. Scores are divided by $\sqrt{d}$ (where d is the dimension per head) to stop the values from growing too large and squashing the softmax gradients.

4. A softmax turns the scores for each token into weights that sum to 1.

5. Each token's output is the weighted sum of all Value vectors, using those weights:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Intuition: if the word "it" is attending strongly to the word "dog" earlier in a sentence, the attention weight between those two tokens will be high, and "it"'s output vector will absorb information from "dog"'s value vector.

Q: What is Multi-Head Attention, and why use several heads instead of one?

A single attention computation can only capture one kind of relationship at a time. Multi-head attention splits the embedding into several smaller chunks (e.g. 1152 dimensions split into 16 heads of 72 dimensions each in SigLIP) and runs attention independently on each chunk. One head might learn to track colour similarity, another shape, another spatial distance. The outputs of all heads are concatenated and projected back to the original dimension.

Q: What is the Encoder in a Transformer?

The encoder is the half of the Transformer that *reads and understands* the input — it converts an input sequence (tokens, or image patches) into a set of context-aware feature vectors. Each encoder block contains: self-attention (so every token gathers relevant information from all others) followed by a feed-forward MLP (so each token individually transforms that information), with residual connections and LayerNorm around both. SigLIP, DINOv2 and EVA-02 in this project are all *encoder-only* models — they only need to understand an image, not generate a new sequence, so no decoder is required.

Q: What is the Decoder in a Transformer, and how is it different from the encoder?

The decoder is the half of the Transformer that *generates* an output sequence one token at a time (used in translation, text generation, captioning, etc.). It differs from the encoder in two ways:

1. **Masked self-attention** — when generating token t, the decoder is only allowed to attend to tokens $1 \dots t-1$ (it cannot see the future, since those tokens haven't been generated yet).
2. **Cross-attention** — after masked self-attention, the decoder attends a second time, but here the Queries come from the decoder while the Keys and Values come from the encoder's output. This is how the decoder "looks back" at the input while generating each output token.

A full encoder–decoder Transformer (e.g. the original 2017 translation model, or the RAG answer-generation model used later in this project) uses both halves. A pure classification/feature-extraction model like SigLIP only needs the encoder half.

Q: Where do Encoder and Decoder actually appear in this project?

- **Encoder-only:** SigLIP-SO400M, DINOv2-L, and EVA-02-L — all three backbones only encode an image into a feature vector; there is no generation step, so no decoder is needed.
- **Decoder (language model):** the RAG answer-generation stage (`gpt-4.1-mini`) is decoder-based — it generates the nutrition-question answer token by token, conditioned on the retrieved documents.
- **No encoder–decoder pair together:** nothing in this project uses a full encoder–decoder Transformer in one model; the image side is encoder-only and the language-generation side is decoder-only, connected via the RAG retrieval pipeline rather than cross-attention.

Q: Why does a Vision Transformer need patches instead of feeding raw pixels?

Self-attention cost grows quadratically with the number of tokens (729^2 pairwise comparisons for 729 patches). Feeding every pixel as its own token (384×384 = 147,456 pixels) would be computationally impossible. Cutting the image into 14×14 patches and treating each patch as one token reduces 384×384 pixels down to a manageable 729 tokens, each carrying a small neighbourhood of visual information.

Q: Why is a residual connection (the "+" around each block) necessary?

Each sub-layer's output is added back to its input: $x + f(x)$. With 27 stacked layers, gradients would otherwise shrink toward zero as they propagate backward through so many layers (vanishing gradients), making early layers very hard to train. The residual path creates a direct shortcut for gradients to flow backward undiminished, alongside the transformed signal.

7 Dataset, EDA and Visualization — Viva Questions

These are general questions your examiner may ask about the dataset and exploratory analysis, independent of SigLIP specifically. Answered in plain text, grounded in the project report.

Q: What dataset was used, and how big is it?

Food-101: 101 dish categories, 101,000 images total, exactly 1,000 images per class (750 train / 250 test by the original authors' split). It was introduced by Bossard, Guillaumin and Van Gool at ETH Zürich (ECCV 2014), collected from the food-sharing platform Foodspotting.

Q: Why is class balance important, and is this dataset balanced?

Food-101 is perfectly balanced by construction — every one of the 101 classes has exactly 750 training and 250 test images, so the imbalance ratio is exactly 1.0 and the standard deviation of class counts is 0. This matters because with a balanced dataset: (1) plain accuracy is a fair, undistorted headline metric, (2) no resampling, class weighting, or SMOTE is needed, and (3) macro-averaged and weighted-averaged metrics coincide exactly — which doubles as a free sanity check on the evaluation code.

Q: How was the validation set created, since Food-101 only ships train and test?

4% of the training split (3,030 images) was carved out as validation, leaving 72,720 images for weight updates, and the 25,250-image test split was left completely untouched until final evaluation. The split is class-stratified (exactly 30 images held out per class) and seeded (`SEED = 1337`) so it is reproducible, and the same split function is shared between the image pipeline and the cached-feature pipeline so both agree on exactly which images are held out.

Q: Why does it matter that the same split function is shared between pipelines?

If the image pipeline and the feature-caching pipeline disagreed even slightly about which images were "validation," validation images would silently leak into training. This would inflate every downstream number — accuracy, the calibration temperature, and the conformal quantile — without producing any visible error. This is called a *deterministic, class-stratified split*, and its correctness was treated as a first-class engineering concern rather than an afterthought.

Q: What EDA (exploratory data analysis) was performed, since this is image data rather than tabular data?

Because there are no feature columns to correlate, EDA for images focuses on different things: class balance, physical image properties (size, aspect ratio), colour/exposure statistics, visual inspection of samples, duplicate/leakage detection between train and test, and class separability in embedding space. Six such analyses were performed and each one changed a concrete decision in the pipeline.

Q: What did the class-balance check show, and what decision came from it?

Counting files confirmed exactly 750 train / 250 test images per class with zero variance. Decision: no resampling, undersampling, or class weighting was needed, and macro/weighted metrics were expected to be numerically identical — which was later confirmed and used as a correctness check.

Q: What did the image-property analysis show?

Every image has its longest side capped at exactly 512 px, confirming the dataset creators pre-resized the collection. About 61.6% of images are square, 26.0% landscape, 12.5% portrait — so 38.5% are non-square. Decision: since backbones require a fixed square input (384/448/518 px), a resize-and-crop step is unavoidable and will discard part of non-square frames — this directly

motivated using `RandomResizedCrop` during fine-tuning, turning an unavoidable information loss into a source of useful training variation.

Q: What did the colour/exposure analysis show?

Food photography in this dataset is noticeably "warm": mean red channel intensity 0.547 versus mean blue 0.345, a gap of 0.20 — much larger than the gap in the ImageNet statistics (under 0.08) that the pretrained backbones were normalised under. Decision: this justified using `color_jitter=0.3` during fine-tuning, since the real deployment case (phone camera, unknown lighting) should not let the model depend on the particular white balance of restaurant photography.

Q: What was the purpose of the duplicate/train-test leakage check, and what did it find?

If test images were near-duplicates of training images, test accuracy would be artificially inflated and untrustworthy. Using cached SigLIP embeddings, the nearest training neighbour was found for every one of the 25,250 test images by cosine similarity. Only 0.09% of test images had a training neighbour above a 0.999 similarity threshold. Decision: no de-duplication was necessary, and the test accuracy could be trusted as an honest estimate.

Q: What was the "class separability" analysis, and why is it called the most useful EDA result?

Before any classifier was trained, a centroid was computed for each class in SigLIP embedding space, and a "separability margin" was defined as within-class tightness minus similarity to the nearest other class. A small margin predicts a class will be hard to classify. This prediction was then compared against the trained model's actual per-class results: 4 of the 5 classes predicted hardest from geometry alone (e.g. `steak`, `filet_mignon`) turned out to be among the model's 5 actual worst classes. It is called the most useful result because it shows the residual error is a property of the label taxonomy itself (some dish categories genuinely overlap visually), not a deficiency the training procedure could have fixed — and it directly motivated adding conformal prediction sets for exactly these ambiguous cases.

Q: What visualizations were produced during EDA, and what does each show?

- *Per-class accuracy histogram* — shows most classes cluster near 97–100% accuracy with a long left tail of a few harder classes; the mean alone would hide this tail.
- *Images-per-class bar charts (train/test)* — perfectly flat bars, visually confirming balance.
- *Width/height/aspect-ratio histograms* — show the 512 px cap and the spread of aspect ratios.
- *Per-channel colour histograms (R/G/B), brightness and contrast histograms* — show the warm colour bias relative to ImageNet.
- *Grid of random sample images* — a manual visual sanity check; the report notes statistics can hide problems that are obvious on sight.
- *Train/test nearest-neighbour similarity histogram* — shows the distribution is centred well below 1.0, supporting the no-leakage conclusion.
- *Class-separability margin histogram and confusion-prone class table* — shows which classes are predicted to be hard before training.
- *PCA projection of class centroids* — a 2D scatter plot showing that desserts, salads, soups and meat dishes form loose neighbourhoods, and that the specific class pairs which later cause confusion errors sit almost on top of each other in the projection.

Q: Why was PCA chosen over t-SNE for visualizing class centroids?

PCA is linear and deterministic, so distances between points in the plot remain mathematically interpretable (a large distance in the plot reliably means a large distance in the original space). t-SNE is non-linear and can distort or fabricate apparent clusters/distances, which would undermine the plot's use as evidence rather than decoration.

Q: What data quality issues does the dataset have, and how were they handled?

The Food-101 authors themselves state the training split was deliberately left uncleaned — it contains some mislabelled images and colour-cast artefacts, while the test split was manually cleaned. Rather than trying to manually relabel 72,720 images, this was handled with *label smoothing* ($\varepsilon = 0.1$) and *Mixup*, both of which reduce the damage a confidently wrong label can do by preventing the model from driving any single target probability all the way to 1.0.

Q: Were there missing values or duplicate images to clean?

No missing values — every record has both an image and a label, so no imputation was needed. No exact duplicate files were found across all 101,000 images; near-duplicates may exist in the original Foodspotting source, but de-duplication was outside the compute budget and was not necessary given the leakage check results above.

Q: Why does class imbalance not need to be corrected here, and what would you have done if it existed?

It does not need correcting because Food-101 is exactly balanced (zero standard deviation in class counts). If imbalance had existed, appropriate responses would include class-weighted loss, oversampling the minority classes, undersampling the majority classes, or reporting macro-averaged metrics (which treat every class equally) instead of relying on plain accuracy, which would otherwise be dominated by the majority classes.

Q: What is the difference between macro-averaged and weighted-averaged metrics, and why do they coincide here?

Macro-averaging computes a metric (precision/recall/F1) separately for each class and then takes an unweighted mean across classes — every class counts equally regardless of size. Weighted-averaging weights each class's contribution by its number of samples. Because Food-101's test set has exactly 250 images per class, every class already carries equal weight, so the two averaging schemes produce numerically identical results — and the report uses this exact coincidence as a built-in correctness check on the evaluation code.

8 Likely Defense / Viva Questions — Prepared Answers

title=Why not fine-tune SigLIP instead of just probing it?

Compute cost. Fine-tuning would cost hours per epoch on the M3 Pro versus minutes for a probe on cached features. The evidence (EVA-02's fine-tune underperforming its own probe) suggests full fine-tuning on only 72,720 images risks overfitting a 400M+ parameter model — frozen features from web-scale pretraining were already strong enough.

title=What does "contrastive" mean, and why is it good for food images?

The model was trained to match images to their natural-language captions from web data rather than to fixed class labels. This gives it broad semantic knowledge of "what things are called," which transfers well to a food-naming task even before any Food-101-specific training.

title=What is the concrete difference between SigLIP and CLIP?

Loss function. CLIP uses softmax over the whole batch (every image competes against every caption in the batch — needs batch-wide computation). SigLIP uses an independent sigmoid per (image, text) pair — no batch-wide normalization required, which is cheaper and more stable at scale.

title=Why is the embedding dimension 1152 instead of the "normal" 1024?

It is a byproduct of the Shape-Optimized (SO) architecture search — depth, width, and MLP ratio were jointly tuned for compute efficiency rather than following the standard ViT-L/ViT-H convention, which is why the numbers (27 layers, 1152 dim) look non-standard.

title=Could SigLIP alone have been used, skipping the ensemble?

Yes, and it would have scored 96.83%. But a McNemar test on the ensemble versus the best single model showed the 0.33-point gain to 97.16% was statistically significant ($p = 3 \times 10^{-6}$), so the extra model was worth keeping.

title=What is L2-normalization and why does it matter for SigLIP embeddings?

Before cached embeddings are fed into the probe head, each vector is divided by its own norm so all vectors have unit length — removing scale differences between images. This step must be identical at training and serving time; skipping it does not crash anything, it silently degrades accuracy, which is the hardest kind of bug to catch.

Prepared as a supplementary technical note for the FoodGenome AI project report. All figures are drawn directly from the project's own reported results (Sections 4–6 and Appendix A of the main report).